

Disciplina:
Arquiteturas de Softwares

Prof. Esp. Leonardo Dias
Curso de Sistemas de Informação

Padrões de Design

Dia 06

Disciplina: Arquiteturas de Softwares

Arquitetura não é só “um desenho bonito”, mas sim a estrutura que sustenta o software.

Padrões de Design

1. Introdução aos Padrões de Design

1.1 O que são Padrões de Design?

- **Definição:**
São **soluções gerais, reutilizáveis e testadas** para problemas recorrentes no desenvolvimento de software.
Não são um código pronto, mas uma **forma de pensar e estruturar** o código.
- **Origem:**
 - O termo vem da Arquitetura Civil, com Christopher Alexander (anos 1970).
 - Aplicado ao software pelo grupo conhecido como **Gang of Four (GoF)** em 1994 no livro *Design Patterns: Elements of Reusable Object-Oriented Software*.

Analogia:

Um padrão de design é como uma **receita de bolo**:

- A receita não é o bolo pronto (não é código pronto).
- Mas fornece um **passo a passo validado** que pode ser seguido e adaptado.
- O bolo pode ter recheios diferentes → assim como o padrão pode ser aplicado em diferentes linguagens e contextos.

1.2 Por que usar?

1. Reduz Acoplamento

Problema: quando duas coisas dependem demais uma da outra, qualquer mudança gera confusão.

- **Analogia:**
Imagine uma **TV antiga** que só funciona com um controle específico da mesma marca.
Se o controle quebrar, você fica sem opção.

Agora pense nas **TVs modernas** que aceitam qualquer controle universal.

- A TV (sistema) continua funcionando sem precisar mudar, e você pode trocar o controle (dependência) facilmente.

O padrão faz o papel do **controle universal**, reduzindo a dependência entre as partes.

2. Aumenta a Reutilização

Problema: você precisa reaproveitar uma solução em diferentes contextos.

- **Analogia:**
Um **carregador USB-C** pode ser usado para carregar o celular, o notebook e até um fone de ouvido.
Já um carregador proprietário só serve para aquele modelo de celular.

Usar padrões é como adotar o **USB-C**: você cria soluções que podem ser aplicadas em vários lugares, sem precisar reinventar.

3. Facilita Manutenção

Problema: sistemas mudam com o tempo, e o código precisa acompanhar.

- **Analogia:**
Pense em um **prédio bem projetado** com fiação organizada em conduítes.
Se precisar trocar a rede elétrica, basta puxar novos fios pelo conduíte.

Agora imagine um prédio com fiação improvisada, passando por dentro das paredes sem padrão.

Trocar qualquer fio é uma dor de cabeça.

O padrão de design é como o **conduíte**: ele organiza o fluxo de forma previsível, tornando mudanças e manutenções muito mais fáceis.

4. Cria Linguagem Comum entre Desenvolvedores

Problema: comunicação em equipe sem padrões é lenta e confusa.

- **Analogia:**
Quando dois médicos conversam, basta um dizer "paciente apresenta quadro de **hipertensão**" que o outro já entende todo o contexto clínico.
Se fosse uma conversa sem linguagem comum, ele teria que explicar: "a pressão está constantemente acima de 14 por 9, pode causar riscos cardiovasculares..."

Os padrões funcionam como essa **linguagem técnica comum**: se um dev fala "vamos aplicar um **Observer** aqui", todos já entendem a solução sem precisar explicar linha por linha.

1.3 Classificação dos Padrões

Os padrões de design são tradicionalmente classificados em 3 grandes grupos:

1. Criacionais – focam na criação de objetos

- Resolvem **como instanciar objetos** sem acoplamento forte.
- Exemplos: **Singleton, Factory Method, Abstract Factory, Builder, Prototype.**

Analogia:

Pense em um **chefe de cozinha (Factory)** que cria diferentes pratos sob demanda, em vez de você mesmo precisar saber todos os detalhes da receita.

2. Estruturais – focam na composição de classes e objetos

- Lidam com **como os objetos se relacionam e se organizam.**
- Exemplos: **Adapter, Facade, Composite, Proxy, Decorator.**

Analogia:

Um **Adaptador de tomada**: você tem um plugue americano e uma tomada brasileira → o adaptador resolve a incompatibilidade.
No software, o *Adapter* faz diferentes sistemas se “encaixarem”.

3. Comportamentais – focam na comunicação entre objetos

- Lidam com a **interação e distribuição de responsabilidades.**
- Exemplos: **Observer, Strategy, Command, State, Chain of Responsibility.**

Analogia:

Um **jornal** que publica notícias para assinantes.

- Quando o jornal tem uma novidade (evento), todos os assinantes (observers) recebem a informação.
- Esse é exatamente o padrão **Observer**.

1.4 Onde se encaixam nas Arquiteturas de Software?

- **Monólitos:**
 - Padrões evitam que o código fique “espaguete” e difícil de manter.
 - Exemplo: usar **Facade** para criar uma interface única em módulos grandes.
- **Distribuídas / Microsserviços:**
 - Padrões são aplicados dentro de cada serviço e também na comunicação entre eles.
 - Exemplo: **Strategy** para implementar múltiplos métodos de autenticação em um microsserviço de usuários.
- **Contêineres (Docker/Kubernetes):**
 - Padrões de design ajudam a manter o código coeso e previsível, o que facilita a escalabilidade em ambientes containerizados.
 - Exemplo: **Singleton** para conexões de banco evita que cada container crie múltiplas conexões desnecessárias.

1.5 Diferença entre Padrão de Projeto e Padrão Arquitetural

- **Padrão de Projeto (Design Pattern):**
 - Foco em **nível de código** (criação, organização e comunicação de classes/objetos).
 - Exemplo: Observer, Singleton.
- **Padrão Arquitetural:**
 - Foco em **nível de sistema** (como organizar módulos, camadas e componentes).
 - Exemplo: MVC, Microkernel, Microsserviços.

Analogia:

- Padrão Arquitetural = **planta da casa** (como os cômodos se organizam).
- Padrão de Projeto = **design dos móveis** dentro dos cômodos (como cada parte funciona e se relaciona).

Ecosistema JavaScript

No ecossistema JavaScript (tanto **frontend** quanto **backend**), alguns padrões de design aparecem com bastante frequência, justamente porque ajudam a organizar código, reduzir acoplamento e tornar o sistema mais manutenível e escalável.

Padrões de Design Mais Usados em JavaScript Frontend

(React, Angular, Vue, etc.)

1. Module Pattern

- Usado para encapsular lógica e evitar poluição do escopo global.
- Exemplo: um módulo de validação de formulários ou de manipulação de API.
- Hoje, ES Modules (import/export) são a evolução natural desse padrão.

2. Observer Pattern (ou Publish/Subscribe)

- Muito usado em sistemas de eventos no navegador (ex: `addEventListener`) e em bibliotecas de estado (Redux, Vuex).
- Permite que componentes "observem" mudanças e reajam sem acoplamento direto.

3. Factory Pattern

- Cria objetos sem expor a lógica complexa de criação.
- Exemplo: uma função que cria componentes de UI dinamicamente a partir de configurações.

4. Singleton Pattern

- Garante que haja apenas uma instância de algo (ex: gerenciador de tema, serviço de autenticação).
- Em apps React/Angular, muitas vezes implementado como "service" compartilhado.

5. Strategy Pattern

- Permite trocar algoritmos em tempo de execução.
- Exemplo: um validador de senha que pode usar diferentes estratégias (mínimo de caracteres, regex, lista de palavras proibidas).

6. Decorator Pattern

- Usado em frameworks como Angular e NestJS (`@Component`, `@Injectable`).
- Em React, o conceito de **higher-order components (HOC)** e **hooks** também lembra decorators.

Padrões de Design Mais Usados em JavaScript Backend

(Node.js, Express, NestJS, etc.)

1. **Singleton**

- Um banco de dados, por exemplo, deve ter uma única conexão reutilizada (MongoDB, PostgreSQL, etc.).

2. **Factory**

- Criação de serviços ou middlewares de forma flexível.
- Exemplo: uma função que retorna diferentes adaptadores de cache (Redis, memória).

3. **Observer / Pub-Sub**

- Muito usado com WebSockets, eventos de sistema (EventEmitter no Node).
- Exemplo: envio de notificações quando uma ordem de compra é finalizada.

4. **Proxy**

- Intercepta chamadas e adiciona lógica extra.
- Exemplo: proxy reverso de APIs, middleware de autenticação em Express.

5. **Command Pattern**

- Encapsula uma ação em um objeto.
- Exemplo: fila de jobs em background (Bull, RabbitMQ) onde cada job é um "comando".

6. **Repository Pattern** (muito comum no backend, inclusive em JS/TS)

- Cria uma camada de abstração entre a aplicação e o banco de dados.
- Exemplo: UserRepository com métodos findByEmail, create, etc.

7. **Adapter Pattern**

- Permite conectar código existente com novas APIs sem reescrever tudo.
- Exemplo: adaptar diferentes provedores de pagamento para uma interface única.

Em ambos (frontend e backend)

- **MVC (Model-View-Controller)** → Base de muitos frameworks.
- **MVVM** (muito usado em Angular, Vue, Knockout).
- **Dependency Injection** (NestJS, Angular, inversão de controle em geral).

Em resumo:

- **Frontend** costuma usar muito **Observer, Module, Decorator, Strategy e Singleton**.
- **Backend** tende a usar mais **Singleton, Repository, Factory, Observer/Pub-Sub, Adapter, Command**.